

## ● PARTE 2: ENTREGA DA AV2 (Qualidade, Arquitetura e Operação)

### O que você vai programar na AV2

Na AV1 você provou que sua API compila, conecta ao banco e faz operações básicas (CRUD). Agora, na AV2, você vai evoluir o sistema com complexidade real de negócio e vai produzir o **SDD (Software Design Document)** — o conjunto de artefatos que prova que o sistema foi *projetado* com Engenharia de Software, e não apenas "codado".

### Requisitos de Código (agnósticos de domínio)

Com base nas Histórias de Usuário do seu backlog, implemente na sua Minimal API:

1. **Pelo menos 2 novos endpoints** (ou evoluções significativas de endpoints existentes) que implementem regras de negócio do seu domínio — não podem ser apenas CRUD simples de 1 tabela.
2. **Pelo menos 1 endpoint deve envolver JOIN:** Uma consulta SQL com `INNER JOIN` ou `LEFT JOIN` que retorne dados relacionados de duas ou mais tabelas.
3. **Pelo menos 1 endpoint deve aplicar múltiplas validações de negócio:** Antes de executar o `INSERT/UPDATE` no banco, o código C# deve verificar pelo menos 3 condições de negócio. Se qualquer validação falhar, a API deve retornar `400 Bad Request` com mensagem específica.
4. **Todas as queries continuam parametrizadas:** Assim como na AV1, nenhuma concatenação ou interpolação de string em comandos SQL. Use exclusivamente `@Parametro` no Dapper.

### O que NÃO é avaliado na AV2

A AV2 **não avalia** se o domínio do seu sistema é "criativo" ou "complexo". Um sistema de agenda de consultas, um controle de tarefas, um marketplace ou um catálogo de filmes — todos têm o mesmo peso. O que importa é a **qualidade da engenharia** aplicada sobre ele, materializada nos 20 artefatos abaixo.

---

## Tabela de Avaliação AV2 (10 Pontos = 20 itens × 0,5)

Os 20 itens abaixo compõem o SDD do seu projeto. Cada item é verificado automaticamente pela presença de padrões específicos nos arquivos do repositório. Leia com atenção: o que está entre `` é o literal que a IA vai buscar.

| #  | O que o grupo DEVE FAZER para ganhar 0,5 ponto                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Onde estudar |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 01 | <b>Padrão AAA nos Testes:</b> Na pasta <code>/tests</code> , os métodos de teste devem conter os comentários <code>// Arrange</code> , <code>// Act</code> e <code>// Assert</code> separando visualmente as 3 fases. Pelo menos 3 métodos de teste (entre todos os arquivos) precisam ter os 3 comentários.                                                                                                                                                                                                                                                                                                        | Cap. 10      |
| 02 | <b>Nomenclatura e Independência de Testes:</b> O nome de cada método de teste deve seguir o padrão de 3 partes separadas por underscore:<br><code>Metodo_Cenario_ResultadoEsperado</code> (ex:<br><code>CancelarPedido_QuandoStatusForAguardandoPagamento_DeveMudarParaCancelado</code> ). Além disso, nenhum método de teste pode conter desvios condicionais ( <code>if</code> , <code>switch</code> , <code>for</code> , <code>foreach</code> , <code>while</code> ). Teste com condicional é frágil e zera o item.                                                                                              | Cap. 10      |
| 03 | <b>Análise de Padrões Arquiteturais:</b> Criar o arquivo <code>/docs/analise_arquitetura.md</code> . Na primeira parte, identificar o padrão arquitetural provável para 3 cenários fornecidos e, para cada um, descrever pelo menos 1 trade-off (um ponto positivo e um negativo) usando os termos <code>Trade-off:</code> ou <code>Positivo:</code> e <code>Negativo:</code> .                                                                                                                                                                                                                                     | Cap. 11      |
| 04 | <b>Análise de Violações Arquiteturais:</b> No mesmo arquivo <code>/docs/analise_arquitetura.md</code> , analisar um trecho de código C# fornecido e listar pelo menos 5 violações arquiteturais. Cada violação deve usar o formato exato:<br><code>**Problema:**</code> , <code>**Evidência:**</code> , <code>**Impacto:**</code> e <code>**Ação Recomendada:**</code> .                                                                                                                                                                                                                                            | Cap. 12      |
| 05 | <b>ADR (Architecture Decision Record):</b> Criar pelo menos 1 arquivo na pasta <code>/docs/adrs/</code> (ex: <code>001-escolha-do-micro-orm.md</code> ) contendo os cabeçalhos <code>## Contexto</code> , <code>## Decisão</code> e <code>## Consequências</code> . O ADR deve conter o campo <code>Status:</code> com um valor entre <code>Proposto</code> , <code>Aceito</code> , <code>Rejeitado</code> ou <code>Obsoleto</code> . Na seção <code>## Consequências</code> , incluir listas separadas com <code>Prós:</code> (ou <code>Positivas:</code> ) e <code>Contras:</code> (ou <code>Negativas:</code> ). | Cap. 13      |
| 06 | <b>Registro de Dívida Técnica (Tabela):</b> Criar o arquivo <code>/docs/registro_divida_tecnica.md</code> com uma tabela Markdown contendo as colunas <code>ID da Dívida</code> , <code>Descrição Técnica</code> , <code>Freq. Alteração</code> , <code>Risco</code> , <code>Esforço</code> e <code>Decisão</code> . Listar pelo menos 6 dívidas reais do próprio código. As colunas <code>Freq. Alteração</code> , <code>Risco</code> e <code>Esforço</code> devem usar valores padronizados ( <code>Alto</code> , <code>Médio</code> , <code>Baixo</code> ).                                                      | Cap. 14      |
| 07 | <b>Priorização de Dívida Técnica:</b> Na mesma tabela do item 06, a coluna <code>Decisão</code> deve classificar cada dívida com um dos 3 níveis: <code>Prioridade 1 (Imediato)</code> , <code>Prioridade 2 (Próxima Sprint)</code> ou <code>Prioridade 3 (Aceitar/Ignorar)</code> . Pelo menos uma dívida deve ser <code>Prioridade 1</code> e pelo menos uma <code>Prioridade 3</code> .                                                                                                                                                                                                                          | Cap. 14      |
| 08 | <b>Classificação de Manutenção:</b> Criar o arquivo <code>/docs/fluxo_manutencao.md</code> . Na primeira parte, classificar 12 tickets fictícios fornecidos usando a taxonomia de                                                                                                                                                                                                                                                                                                                                                                                                                                   | Cap. 15      |

| #  | O que o grupo DEVE FAZER para ganhar 0,5 ponto                                                                                                                                                                                                                                                                                                                                                        | Onde estudar |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
|    | Swanson: Corretiva , Adaptativa , Perfectiva ou Preventiva . Cada ticket deve ter a classificação explicitamente anotada (ex: Ticket 1 → Corretiva ).                                                                                                                                                                                                                                                 |              |
| 09 | <b>Pipeline de Liberação Segura:</b> No mesmo arquivo <code>/docs/fluxo_manutencao.md</code> , descrever o pipeline de liberação em segurança para um ticket de correção, documentando os 4 passos: 1. Análise de Impacto , 2. Teste como Instrumento Cirúrgico , 3. Feature Toggle e 4. Estratégia de Release e Regressão .                                                                          | Cap. 15      |
| 10 | <b>Plano de Iteração:</b> Criar o arquivo <code>/docs/plano_iteracao.md</code> contendo os campos: Objetivo da Iteração: , Escopo (Backlog Selecionado): , Entregáveis (Evidências): , Risco Principal do Ciclo: e Definição de Pronto (DoD): . Todos os campos devem estar preenchidos com conteúdo específico do projeto.                                                                           | Cap. 16      |
| 11 | <b>Quadro Visual e Limite de WIP:</b> No mesmo arquivo <code>/docs/plano_iteracao.md</code> , descrever um quadro visual com pelo menos 4 colunas nomeadas (ex: Backlog , Em Desenvolvimento , Code Review , Concluído ). Incluir explicitamente um <b>limite de WIP numérico</b> (ex: WIP máximo: 3 tarefas ). O número do WIP deve ser menor ou igual ao número de integrantes do grupo.            | Cap. 16      |
| 12 | <b>Matriz de Riscos:</b> No arquivo <code>/docs/operacao.md</code> , criar uma tabela Markdown com as colunas Risco , Probabilidade , Impacto , Estratégia e Ação Planejada . Listar pelo menos 5 riscos. As colunas Probabilidade e Impacto devem usar valores padronizados ( Alto , Médio , Baixo ). A coluna Estratégia deve conter um dos 4 tipos: Mitigar , Transferir , Aceitar ou Evitar .     | Cap. 17      |
| 13 | <b>Gatilhos de Risco:</b> Na mesma tabela do item 12, incluir a coluna Gatilho . Todas as linhas devem ter a célula de gatilho preenchida com pelo menos 20 caracteres, descrevendo o evento observável e irrefutável que dispara a ação.                                                                                                                                                             | Cap. 17      |
| 14 | <b>Métrica de Fluxo (DORA):</b> No arquivo <code>/docs/operacao.md</code> , definir uma primeira métrica operacional no formato de Ficha de Definição Operacional com os 7 campos: Nome da Métrica: , O que Mede: , Fórmula: , Fonte de Dados: , Frequência de Coleta: , Limites de Saúde: e Ação se Violado: . A métrica deve ser de fluxo (ex: mencionar Deploy , Lead Time , Throughput ou DORA ). | Cap. 18      |
| 15 | <b>Métrica de Qualidade:</b> No mesmo arquivo, definir uma segunda métrica com os mesmos 7 campos do item 14. A métrica deve ser de qualidade (ex: mencionar Falha , Erro , Teste , Change Failure Rate ou Cobertura ).                                                                                                                                                                               | Cap. 18      |
| 16 | <b>SLO (Service Level Objective):</b> No arquivo <code>/docs/operacao.md</code> , definir 1 SLO para a rota mais crítica do sistema. Deve conter: SLI (Indicador): , Fórmula de Coleta: , Fonte do Dado: , Janela de Medição: com valor numérico seguido de dias ou horas , e Alvo (SLO): com um valor de porcentagem (ex: 99.5%).                                                                    | Cap. 19      |

| #  | O que o grupo DEVE FAZER para ganhar 0,5 ponto                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Onde estudar |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 17 | <b>Error Budget Policy:</b> No mesmo arquivo, abaixo do SLO, definir a <code>Error Budget Policy</code> : com pelo menos 3 níveis de resposta graduada ( <code>Nível 1</code> , <code>Nível 2</code> , <code>Nível 3</code> ). O último nível ( <code>Nível 3</code> ) deve conter os termos <code>congelamento</code> , <code>Feature Freeze</code> , <code>Zero novas funcionalidades</code> ou equivalente, deixando claro que o desenvolvimento de features para quando o orçamento de erro se esgota.                                                                                         | Cap. 19      |
| 18 | <b>Segurança no Código (SSDF):</b> Nenhum arquivo <code>.cs</code> na pasta <code>/src</code> pode conter strings de conexão ou credenciais hardcoded. São proibidos: <code>Password=</code> , <code>Pwd=</code> , <code>User Id=</code> ou <code>ConnectionString=</code> seguidos de valor literal (ex: <code>Password=senha123</code> ). Strings que referenciam <code>builder.Configuration</code> , <code>Environment.GetEnvironmentVariable</code> ou <code>secrets.json</code> são aceitas. Se a IA encontrar credencial hardcoded, zera este item.                                         | Cap. 20      |
| 19 | <b>Threat Model e Gates de Segurança:</b> Criar o arquivo <code>/docs/seguranca_ciclo.md</code> contendo: <b>(a)</b> um Threat Model para a rota de maior risco, com os campos <code>Ativos Protegidos:</code> , <code>Vetor de Ataque Provável:</code> , <code>Falha Arquitetural Potencial:</code> e <code>Controle de Engenharia (Mitigação):</code> ; <b>(b)</b> 3 Gates de segurança numerados ( <code>Gate 1</code> , <code>Gate 2</code> , <code>Gate 3</code> ) que a equipe adotará.                                                                                                      | Cap. 20      |
| 20 | <b>Topologia de Times e DoD Final:</b> Criar o arquivo <code>/docs/topologia_times.md</code> mapeando os 4 tipos de time do Team Topologies ( <code>Stream-aligned</code> , <code>Platform</code> , <code>Enabling</code> , <code>Complicated-Subsystem</code> ) para o contexto do projeto. Além disso, na raiz do repositório, entregar o arquivo <code>release_checklist_final.md</code> com as 7 caixas de seleção marcadas como concluídas ( <code>[x]</code> ): Fundamentos, Produto Mínimo, Evidência de Qualidade, Decisões Documentadas, Evidência de Requisitos, Governança e Segurança. | Cap. 21 e 22 |